

Instrucciones de funcionamiento de la DGX

Entrenamiento e inferencia

Modelos Generativos Profundos · Máster en Inteligencia Artificial · ICAI · Curso 2026-2027

Para la realización de las prácticas y proyectos de este curso, se han modificado el tipo de sesiones disponibles en la DGX, adecuándose a las necesidades que podáis tener tanto para el entrenamiento como para la inferencia.

1. Recursos disponibles

La DGX dispone de 8 GPUs Nvidia H200 de 141GB de VRAM. Se usa el sistema de GPU multi-instancia (MIG), cuyas nuevas instancias disponibles son las siguientes:

- **46 instancias de 1/7 de GPU (16GB)**. Están pensadas para entrenamiento e inferencia de modelos pequeños, especialmente para el desarrollo del agente ARCA.
- **1 instancia de 1/2 de GPU (71GB)**. Pensada para aquellos equipos que quieran realizar *fine-tuning* e inferencia de modelos con mayor capacidad, principalmente para el desarrollo del proyecto final. **Tiene un máximo de 7 días por sesión**. Cuando se lanza un trabajo en esta instancia, queda ocupada y los trabajos que la requieran quedarán en cola de espera.
- **1 instancia de 1/2 de GPU (71GB)**. Controlada por los profesores solamente y sin límite de tiempo. Está pensada para proporcionar un entorno de inferencia común que pueda servir a todos los grupos para:
 - Diseño de aplicaciones de agentes que no requieran un entrenamiento adicional
 - Generación de trazas de razonamiento para el arranque en frío del SFT para el agente ARCA.

2. Servicio de inferencia común

Una instancia gestionada por los profesores está activa de manera indefinida para todos los grupos de la asignatura.

En esta instancia se ha desplegado un servidor de inferencia vLLM con el modelo [Qwen3.8-27B-FP8](#), con capacidades agénticas y de razonamiento bastante avanzadas. El servidor optimiza el servicio de inferencia para servir a múltiples usuarios de manera simultánea con un rendimiento alto (compilación, batching, KV cache, kernel fusion...). Estos son los ajustes y prestaciones más interesante de cara al usuario:

- Longitud de contexto máxima de 65.536 tokens, incluyendo el prompt.
- Máximo de 5 secuencias simultáneas en proceso.
- Alrededor de 50 tokens por segundo, dependiendo de la longitud de contexto y número de requests simultáneas.
- Entre 1 y 7s de tiempo hasta el primer token, dependiendo de la longitud de contexto y número de requests simultáneas

Para acceder a este servicio, basta con apuntar a la URL <http://127.0.0.1:8000> siempre **dentro de la DGX**. El servidor no es accesible desde redes externas. No todos los endpoints están expuestos, estando disponibles:

- GET:/health: check de estado del servidor.
- GET:/v1/models modelos disponibles.
- POST:/v1/chat/completions: generación de texto en formato chat.
- POST:/v1/completions: generación de texto.
- GET:/metrics: check de métricas del servidor

2.1 Ejemplo de uso

A continuación se muestra un ejemplo de uso del endpoint `POST:/v1/chat/completions` con `curl`:

```
curl -X POST http://127.0.0.1:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.8-27b-fp8",
  "messages": [
    {
      "role": "user",
      "content": "Reply with the word READY."
    }
  ],
  "max_tokens": 64,
  "chat_template_kwargs": {
    "enable_thinking": false
  }
}'
```

Nótese el nombre del modelo `qwen3.8-27b-fp8` y el parámetro `chat_template_kwargs`, que permite desactivar la generación de trazas de razonamiento.

Los ajustes de muestreo deberían ser los de por defecto usando el modo con razonamiento.

Para usar el modo sin razonamiento, consultar los parámetros en [la ficha del modelo](#).

Los frameworks de aplicaciones de agentes tienen normalmente soporte para estas APIs tipo OpenAI. Por ejemplo, LangChain permite establecer conexión mediante un cliente definido como sigue:

```
ChatOpenAI(
    model="qwen3.8-27b-fp8",
    base_url="http://localhost:8000/v1",
    api_key="EMPTY",
)
```

donde `api_key` puede ser cualquier valor, ya que el servidor no requiere autenticación.